Sentinel: AI-Based Intrusion Detection with Human Review and a Data Warehouse

Project Report – Data Warehousing and Mining (DWM) Practical

Jai Chavan Jeet Mane Amaan Khan

Guide: Dr. Kavita Shirsat | Department of Computer Engineering, Vidyalankar Institute of Technology, Mumbai

Project link: `https://github.com/JaiChavan040906/IDS`

1. Abstract

Sentinel is a web application that detects attacks in network traffic. A CNN-LSTM model classifies every network flow, and flows that the model is not sure about are sent to a security analyst instead of being decided automatically. The analyst's decisions are saved and can be used to retrain the model after an administrator approves them. All traffic, alerts and reviews are also loaded into a data warehouse, where they can be analysed with OLAP operations. On unseen test data, only 1.48% of flows needed the analyst, and the automatic decisions were 99.999% correct.

2. Problem Statement and Objectives

AI-based intrusion detection systems give a label to every flow, even for attacks they have never seen, and a wrong label can look just as confident as a correct one. The alerts are also stored in an ordinary database, which makes it hard to analyse them by day, attack type or service. The objectives of this project are:

- to detect normal traffic and 12 types of network attacks using a CNN-LSTM model;
- to send only uncertain flows to a security analyst and save the analyst's decisions;
- to improve the model from verified feedback, only after admin approval;
- to build a data warehouse with ETL and analyse the data using OLAP operations.

3. Methodology

1. Dataset. We used the Improved CIC-IDS2017 dataset (five days of lab traffic). After removing duplicate rows and columns such as IP addresses and ports, 1,714,812 flows with 64 features and 13 classes (normal traffic and 12 attack types) remained. The data was split into 70% training, 15% validation and 15% test. The test part was kept aside and used only once, for the final evaluation.

2. Detection models. The main model is a CNN-LSTM network. The 64 features of a flow are read as a sequence and passed through a 1-D convolution (64 filters), max-pooling, a second convolution (128 filters), an LSTM layer (128 units), dropout (0.3) and an output layer with one value per class. It was trained with the AdamW optimiser (learning rate 0.001, batch size 1,024) for up to 20 epochs, stopping early when the validation score stopped improving. Classes with fewer than 1,000 training rows were oversampled so the model did not ignore them. A Random Forest (100 trees) and LightGBM were trained on the same data for comparison, and the Random Forest is also used as a second opinion. Every model was trained with three random seeds.

3. Uncertainty check. A flow is sent to the analyst if the prediction is not confident (high entropy), if the flow is unlike the training data (Mahalanobis distance), or if the Random Forest gives a different answer. All other flows are closed or alerted automatically.

4. Analyst review and retraining. The analyst marks a flow as malicious, benign or unsure and gives a reason. These cases are stored and shown as similar incidents later. After 20 decisions are verified by the admin, a new model can be trained; it is deployed only if it passes fixed checks, and it can be rolled back.

5. Data warehouse and ETL. The ETL job *extracts* the dataset and the portal's alerts and reviews, *transforms* them (time keys, port to service name, attack to category) and *loads* them into a separate SQLite warehouse. The main design is a star schema: the fact table `fact_flow` (one row per flow) with time, attack, service, protocol and split dimensions. Snowflake and fact constellation versions, an hourly aggregate table and two data marts are also built. A full load rebuilds everything; an incremental load adds only new alerts.

6. OLAP. The OLAP page supports roll-up, drill-down, slice, dice and pivot, and shows the result as a table, a chart and a 3-D cube.

4. Results

All detection results were measured on the 15% test data (257,222 flows), which the models never saw during training. Table 1 compares the models (attack vs normal, average of three runs).

Table 1. Model comparison on the test data

Model	Detection rate	Precision	F1 score	Accuracy (13 classes)
Reference paper (de Nascimento and Hou, 2025)	96.6%	92.1%	94.3%	–
Random Forest	99.92%	99.90%	99.91%	99.97%
LightGBM	99.91%	99.84%	99.88%	99.96%
CNN-LSTM (our main model)	99.75%	99.71%	99.73%	99.89%
CNN only (LSTM removed)	99.23%	99.70%	99.46%	99.80%

All our models scored above the reference paper. The Random Forest was slightly better than the CNN-LSTM, and removing the LSTM layer lowered the F1 score, so the LSTM does help. Table 2 shows how the full system (model plus uncertainty check) handled the test traffic, and the warehouse measurements.

Table 2. System and warehouse results

Detection		**Data warehouse**	
Flows sent to analyst	1.48%	Flows in fact table	1,714,812
Model errors caught in review	98.9%	Full ETL load	about 25 s
Accuracy of automatic decisions	99.999%	Incremental load	about 0.3 s
Attacks missed / false alarms	2 / 0	Query on aggregate vs all flows	0.3 ms vs 2.3 s

Table 3 shows the OLAP operations performed on the flow cube of the warehouse and what each one showed about the traffic.

Table 3. OLAP operations on the flow cube (measure: number of flows)

Operation	What was done	Result
Roll-up	Attack class → category, per day	Monday has no attacks; Wednesday mostly DoS (177,506)
Drill-down	Wednesday: day → hour	158,662 DoS flows fall in one hour
Slice	Service = HTTP	94.4% of all attack flows used HTTP
Dice	Wed–Thu, DoS/DDoS, HTTP/HTTPS	a smaller sub-cube of web flooding attacks
Pivot	Swap rows and columns	same numbers seen from the other dimension

Tools used: Python, PyTorch, scikit-learn, LightGBM, pandas, FastAPI, SQLite, React and Recharts. The full source code is available at the project link above.

5. Conclusion and Future Work

The system handles most traffic on its own and asks a human only about doubtful flows, which keeps the automatic decisions almost error-free. The data warehouse makes the traffic and alert history easy to explore with OLAP. In future, we plan to detect slow attacks such as SSH brute force by using features over many flows, and to test the system on the CSE-CIC-IDS2018 dataset.